

TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT

Kunda Lin^a, Xiaolong Xu^{b,*}, Honghao Gao^{c,*}

^a Jiangsu Key Laboratory of Big Data Security & Intelligent Processing, Nanjing University of Posts and Telecommunications, Nanjing, China

^b School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing, China

^c School of Computer Engineering and Science, Shanghai University, Shanghai, China

ARTICLE INFO

Keywords:

Encrypted traffic classification
Tor network traffic
Traffic characterization
Deep learning

ABSTRACT

In the Industrial Internet of Things (IIoT) in the 5G era, the growth of smart devices will generate a large amount of data traffic, bringing a huge challenge of network traffic classification, which is the prerequisite of IIoT traffic engineering, quality of service (QoS), cyberspace security, etc. It is difficult for current traffic classification methods to distinguish encrypted dataflow and design effective handcraft features. In this paper, a novel identification scheme of encrypted traffic, TSCRNN, is proposed to automatically extract features for efficient traffic classification, which is based on spatiotemporal features. TSCRNN includes the preprocessing phase and the classification phase. In the preprocessing phase, raw traffic data are processed with flow segmentation, sampling, and vectorization, etc. To solve the classification problem of long time flow, sampling strategies are used to collect samples from the middle of the long-lived flow. In the classification phase, TSCRNN extracts abstract spatial features by CNN and then introduces stack bidirectional LSTM to learn the temporal characteristics. The experiments were performed on the dataset ISCXTor2016. The experimental results show that TSCRNN outperforms other typical methods in all scenarios, which achieves the accuracy up to 99.4% and 95.0% respectively in Tor/nonTor binary classification tasks and sixteen classification tasks. Furthermore, TSCRNN is applied to other real network datasets obtained the satisfactory performance, which validates its feasibility and universality. It means that TSCRNN can effectively identify encrypted and anonymous traffic, provide a fine-grained traffic characterization mechanism, which will support the development of core technologies in the Industrial Internet of Things.

1. Introduction

Traffic classification is the essential task for traffic engineering, network management, quality of service (QoS), and cybersecurity. However, with the popularity of traffic encryption techniques and the rapid growth of network throughput [1], accurate identification of encrypted traffic in real-time is becoming much harder [2]. Recently, demands of traffic analysis have also emerged in new network systems, such as Industrial Internet of Things (IIoT) [3], software-defined network (SDN) [4], and mobile Internet [5]. Therefore, encrypted traffic classification has attracted more attention.

Sensory perception, communication interaction, network interconnection, information processing, and security defense are the core technologies that make up the IIoT. The industry uses a large number of smart sensors to collect data generated in various links of industrial

production, to conduct intelligent data analysis and resource management. However, mass devices will generate extremely huge traffic, which brings great pressure on data communication transmission, resource management, QoS, and causes more security problems [6]. Besides, traffic encryption, obfuscation, and anonymity technologies have become a common practice in the industry [7], which makes it difficult to perceive and manage traffic at the network level. At the same time, security attacks make extensive use of these technologies to evade the interception of firewalls and intrusion detection systems [8]. With the advent of the 5G era, the data collected by sensors will become more abundant, causing these problems to be further aggravated. Therefore, the research on the classification of network encrypted traffic will help strengthen the resource management and security assurance of the IIoT in the 5G era [9].

Typical encrypted traffic classification methods can be divided into

* Corresponding authors.

E-mail addresses: xuxl@njupt.edu.cn (X. Xu), gaohonghao@shu.edu.cn (H. Gao).

<https://doi.org/10.1016/j.comnet.2021.107974>

Received 14 December 2020; Received in revised form 6 February 2021; Accepted 26 February 2021

Available online 3 March 2021

1389-1286/© 2021 Elsevier B.V. All rights reserved.

three main categories: port-based methods, payload-based methods, and statistics-based methods [2,10]. Port-based methods can only achieve low accuracy due to the prevalence of dynamic port and port camouflage techniques. Payload-based detection methods need to match traffic fingerprints, which is time-consuming. At present, more attention is focused on statistics-based methods, which are based on machine learning, and require domain-experts to extract hand-crafted features for a specified scenario. At the same time, offline algorithms are generally required for overall statistical characteristics. Therefore, they usually more time-consuming.

Deep learning has achieved considerable progress in many fields related to classification problems. Convolutional neural networks (CNN) can learn abstract spatial features in data through multi-layer stacking, while recurrent neural networks (RNN) accepts data at multiple time steps and can learn the time characteristics between data. As a typical classification task, deep learning methods can be also applicable to encrypted traffic identification [11]. However, recent studies on the classification of encrypted traffic based on deep learning have not taken into account the packet-to-packet interrelationship. Therefore, in this paper, we combine the stacked bidirectional LSTM (RNN-based architecture) to learn the temporal features of the spatial features extracted by CNN. Furthermore, considering the continuous emergence of new types of network applications, the service types of traffic would be classified (for instance, Chat, P2P, Video, and so on), which is called traffic characterization.

In this paper, a novel identification scheme of encrypted traffic, TSCRNN, is proposed to automatically extract features for efficient traffic classification. The main contributions of this paper are as follows:

- In the preprocessing phase, the original traffic data was subjected to flow segmentation, vectorization, normalization, and other processes to preserve the sequential relationship between packets. Besides, a sampling method was used to collect samples from long flow in the Tor network. TSCRNN used only a small number of packets to realize the early identification of traffic.
- TSCRNN combined CNN and RNN to extract abstract features of the flow at first, and learn the temporal characteristics, to realize efficient identification. Most experiments were conducted on the ISCX-Tor2016 dataset [12]. For the traffic characterization task (sixteen classifications), TSCRNN achieved average accuracy up to 94.8% without handcrafted features design, which is better than both conventional machine learning and deep learning methods.
- TSCRNN was applied to other traffic datasets directly. Without any special configuration, it can still show ideal performance, which indicates that TSCRNN is universal and is not limited to specified scenarios. Experiments show that TSCRNN can effectively identify encrypted and anonymous traffic and achieve fine-grained traffic characterization, which can provide support for network resource management, QoS, and security assurance in the IIoT.

The rest of the paper is organized as follows. Related works about encrypted traffic classification are analyzed in Section 2. In Section 3, we introduce the architecture of TSCRNN, presenting the workflow of the preprocessing and the classification in detail. Section 4 presents the experimental environment and datasets, evaluation matrices, experimental results, and the corresponding analysis. Section 5 concludes this paper and shows the future work.

2. Related work

Typical encrypted traffic classification methods can be divided into three main categories: port-based methods, payload-based methods, and statistics-based methods.

With the rapid development of the network, port-based identification methods of encrypted traffic are no longer suitable for the current network environment [13,14]. The payload-based methods extract byte

patterns of traffic, maintaining a library of traffic fingerprints to identify traffic like string regular matching. The most famous payload-based approach is deep packet inspection (DPI) [15,16]. Those methods are widely used in industry, however, they are time-consuming and the existing patterns are prone to invalidate due to traffic encryption and anonymous technologies [17].

The emergence of machine learning makes up for the deficiency of rule-based methods. Liu et al. [18] designed a series of statistical features at the packet level, such as maximum, minimum, mean, packet size, etc., which were used in a semi-supervised learning framework to classify the encrypted traffic. Anderson et al. [19] used several features, including the meta-information in the header of TLS flow, the distribution of packet length, arrival time, etc., in a logistic regression model to distinguish 18 types of malicious traffic. Fahad et al. [20] chose some features from a features list presented by Moore et al. [21] and introduced the Bayesian kernel estimation method to classify the traffic. Similarly, Yamansavascular et al. [22] referred to the same features list and selected 111 flow characteristics to classify 11 types of traffic applications with the k-nearest neighbor algorithm (k-NN), achieving the 94% of accuracy rate. However, none of them considered the strategy of feature selection. Alshammari et al. [23] chose the features without basic information of traffic (IP, port, etc.) to improve the universality, using the C4.5 decision tree which achieved an accuracy of up to 90%. In [24], the robust traffic classification mechanism (RTC) and the bag of words classifier (BoW) are proposed to classify encrypted traffic. Experiments demonstrated that they perform better and are more robust than other machine learning-based methods. In [25], artificial neural networks and support vector machines were used to dichotomize the public dataset ISCXTor2016 with a 99% accuracy rate. However, it is only suitable for simple coarse-grained binary classifications.

In conclusion, the domain experts are required to extract hand-crafted features, only suitable for specific scenarios. The distribution of traffic flow is used as main feature, which usually requires long traffic collections. It means that a large of temporary traffic needs to be cached and consumes more storage resources.

Deep learning is a branch of machine learning. Compared to the above machine learning-based methods, deep learning-based methods for encrypted traffic classification do not require the design of manual features, which are generally end-to-end. Wang et al. [26] were the first to introduce the one-dimensional convolutional neural networks (1D-CNN) for the VPN-nonVPN dataset [27], which reaches 86.6% accuracy of 12 types of traffic services classification. The 1D-CNN captured byte features at the packet level to realize automatic traffic feature extraction, while temporal characteristics were not considered. The paper [28] presents a traffic classification system, combining multi-level features extracted by both machine learning and deep learning, to identify 4 types of encrypted traffic with high accuracy. As an offline algorithm, it is time-consuming and requires handcrafted feature design as well. The two-dimensional convolutional neural networks (2D-CNN) has been widely applied in image classification. In [29], the FlowPic model is proposed, which transforms a series of packet size in each flow into two-dimensional gray images, and then used CNN to implement the image classification. FlowPic has strong universality as the simple but effective conversion method. However, FlowPic requires long time traffic capturing, which is not suitable for short-lived traffic classification. Also, the author points out that it is not appropriate for tor network traffic classification. The papers [30,31] also combine CNN and RNN to design model architecture. However, Lopez-Martin. et al [30] used manual features to encode traffic without considering the distribution characteristics of traffic bytes. Wang et al [31] considered spatiotemporal features as well, but he proposed a two-stage model to identify network malicious attacks. Each packet is one-hot encoded and inputted to CNN to obtain their respective spatial features at first, and then use RNN to learn the overall timing features.

3. Methodology

3.1. Framework of TSCRNN

TSCRNN is designed to solve the problem of encrypted traffic classification, which includes the preprocessing phase and the classification phase. Fig. 1 shows the framework of TSCRNN. The raw traffic data are converted to the standard formatted data via preprocessing, including flow segmentation, sampling, vectorization, and normalization. In the classification phase, the spatiotemporal features are captured to distinguish traffic types by the mixed neural networks model proposed in this paper. TSCRNN is an end-to-end traffic classification scheme, which outputs the label of traffic service types.

3.2. Preprocessing

3.2.1. Flow segmentation

In a real network system, the traffic collected at a node is not an ordered sequence from a single application. For example, the traffic captured from a gateway may be composed of all packets sent from various hosts on the same network segment. To distinguish the traffic flow produced by a single application, we need to split the raw traffic.

A flow implies that a sequence that contains packets that have the same 5-tuple: (source IP, source port, destination IP, destination port, transport layer protocol). The original traffic should be read, cached and segmented according to the 5-tuple information.

3.2.2. Sampling

As shown in Fig. 2, the flow number of traffic service types in the Tor-nonTor dataset shows the problem of data imbalance (the y-axis is a logarithmic scale). Some types have a large sample size, such as Browsing and P2P; while Tor types generally only have dozens of

samples. The reason for the scarcity of the tor traffic flow is that a virtual circuit may be established to retain a long time session communicating on the tor network, which generates long-lived flows. It is called the onion routing technique proposed for privacy, which encrypts traffic multiple times and hides source addresses.

The paper [32] proposes a sampling method to obtain samples from the elephant flow for semi-supervised learning. Inspired by it, we use an in-flow sampling method to obtain enough samples from Tor long-live flows for supervised learning. The three kinds of sampling strategies are as follows: Random Sampling, the default strategy, randomly selects sampling points in the flow; Fixed Step Sampling, which selects sampling points in a fixed-step; Mixed Sampling, combining the time interval and packet length to find the sampling points in a flow.

In TCP or UDP transmissions, the packets used for interaction without payload are usually less than 60 bytes, Mixed Sampling detects such packets and obtains a series of collection points combining the packet interval time. However, as mentioned in the paper [2], sampling samples from the middle of a flow is still an open problem to be studied. This heuristic method of Mixed Sampling is only for comparison. The default sampling strategy in the experiments is Random Sampling.

3.2.3. Vectorization

Network traffic is binary data. To facilitate the calculation, the traffic is read as bytes, converted to integers from 0 to 255 (8 bits). Each traffic flow contains several time-related packets, we use the first N packets of each flow, the first M bytes of each packet. Fig. 3 shows a vector of flow after vectorization, which contains N packets. Each packet contains M bytes (padding with 0×00).

To select appropriate values of N and M , the distribution of the number of all flows and the distribution of the length of all packets of the Tor-nonTor dataset are counted, as shown in Fig. 4. More than 90% of flows are between 0 and 15, as shown in Fig. 4 (a), while the lengths of almost all packets are in the interval from 0 to 1500 bytes, as shown in Fig. 4 (b). The MTU (Maximum Transmission Unit) of Ethernet is exactly 1500 bytes.

3.2.4. Normalization

To make the model easier to train, all data are normalized by dividing all numbers by 255.

3.3. Classification

The raw traffic data is high dimensional after vectorization. It greatly increases the number of parameters of the deep learning model and computation time, which is more likely to cause overfitting and is harder to train. The sequential relationship between packet to packet is not considered enough.

In this paper, we used CNN to learn the spatial characteristics, so that high-dimensional features could be distilled into abstract feature maps, and then designed a network architecture with RNN to learn the temporal characteristics.

3.3.1. Features extraction

CNN is widely used in the field of deep learning for image processing using 2D-CNN generally. LeCun et al. [33] pointed out that 1D-CNN is suitable for one-dimensional sequence data that has strong local correlations or whose features can appear anywhere. In the network flow, there are specific patterns that could be captured by CNN, and abstract spatial features are extracted through a multi-layer architecture. For example, 1D-CNN [34] was soon used in text data. In the domain of traffic classification, Wang et al. [26] introduced 1D-CNN to converted the traffic flow into a one-dimensional sequence. However, the temporal characteristics of traffic are not preserved, and the timing relationships between packets are lost.

In this paper, the temporal dimension is preserved in the process of

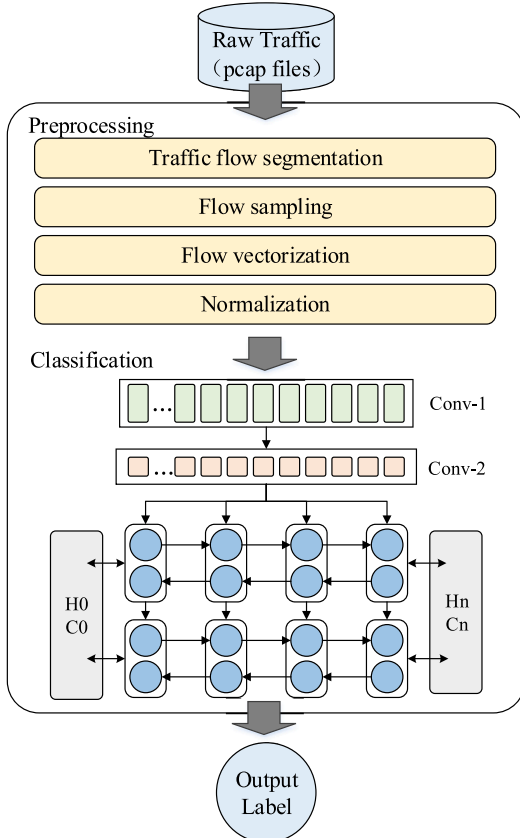


Fig. 1. The framework of TSCRNN.

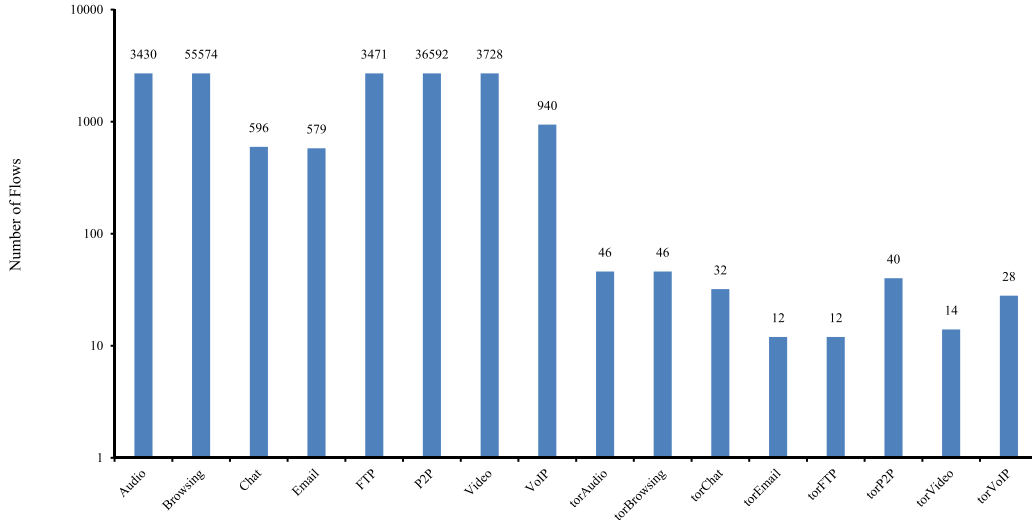


Fig. 2. The flow number of service types in the Tor-nonTor dataset.

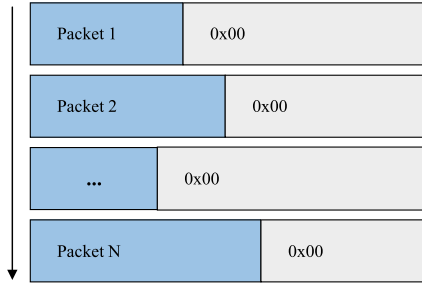


Fig. 3. The vectorization method of a traffic flow.

vectorization. Suppose that $\mathbf{x}^{(t)} \in \mathbb{R}^k$ is a k -dimensional vector corresponding to the t -th packet in a flow.

$$\mathbf{x} = [\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(n)}] \quad (1)$$

where $\mathbf{x}$ represents a flow with n packets. $\mathbf{x}$ is regarded as a two-dimensional vector with n channels, each of which is k -dimension. $\mathbf{x}_{i:i+h-1}$ refers to traffic bytes from i to $i+h-1$ in all channels. A convolutional operation involves the c -th filter $\mathbf{w}^{(c)} \in \mathbb{R}^{hk}$, which is applied to a window of h traffic bytes in all channels to generate a new feature $\mathbf{a}^{(c)}$.

$$\mathbf{a}_i^{(c)} = f(\mathbf{w}^{(c)} \cdot \mathbf{x}_{i:i+h-1} + \mathbf{b}^{(c)}) \quad (2)$$

where $\mathbf{b}^{(c)} \in \mathbb{R}$ is a bias term and f is a non-linear activation function

such as the hyperbolic tangent. The filter is applied to the whole sequence $\{\mathbf{x}_{1:h}, \mathbf{x}_{2:h+1}, \dots, \mathbf{x}_{n-h+1:n}\}$ to generate a feature map $\mathbf{a}^{(c)}$.

$$\mathbf{a}^{(c)} = [\mathbf{a}_1^{(c)}, \mathbf{a}_2^{(c)}, \dots, \mathbf{a}_{n-h+1}^{(c)}] \quad (3)$$

where $\mathbf{a}^{(c)}$ is the feature map of the c -th filter. And then all feature maps generated by m filters will be concatenated to be a new vector $\mathbf{a} \in \mathbb{R}^{mn'}$, where $n' = n - h + 1$ and $\mathbf{a}$ refer to a new feature map, containing m channels. Besides, we use the max-over-time pooling operation to downsample the traffic, which is similar to the convolution operation, but each filter retains the maximum value as a new feature in each operation.

In essence, CNN and fully connected neural networks are similar, while CNN is characterized by filter weight sharing and sparse connection [33], which is of great help for the operation of high-dimensional vectors. Here, traffic data is downsampled by 1D-CNN, and as the layers deepen, the convolution operation will generate more abstract feature maps. It means that the model will learn more advanced features, which will assist subsequent learning of temporal features.

3.3.2. Abstract temporal features learning

RNN plays an important role in the represent learning of time series. The most famous unit structure is Long Short-Term Memory (LSTM) [35, 36], which can solve the problem of gradient vanish (explosion) and has been proved to achieve acceptable detection results in time series analysis. For the same reason, LSTM could be used for network traffic analysis, since it is a high time-dependent data as well.

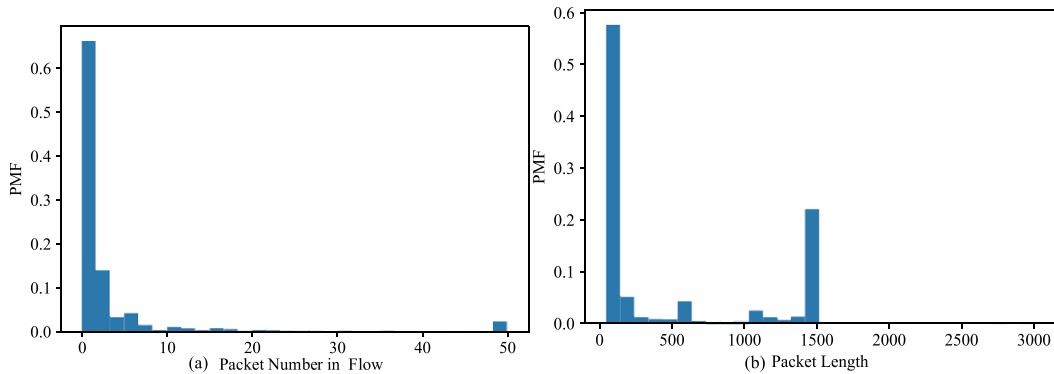


Fig. 4. The statistical distributions of the Tor-nonTor dataset. (a) is the distribution of packet numbers in all flows. And (b) is the distribution of length of all packets.

$$\tilde{c}^{(t)} = \tanh(w_c[h^{(t-1)}, a^{(t)}] + b_c) \quad (4)$$

where $a^{(t)} \in \mathbb{R}^s$, representing the t -th channel of the feature map, which is a s -dimensional vector. $h^{(t-1)} \in \mathbb{R}^u$ is the output of the hidden layer at the last time step, and its dimension is u . w and b are the corresponding parameters matrix and bias term respectively. And $\tilde{c}^{(t)}$ is the intermediate output of the hidden layer. The ultimate output is determined by three terms called gate in LSTM.

$$\Gamma_u = \sigma(w_u[h^{(t-1)}, a^{(t)}] + b_u) \quad (5)$$

$$\Gamma_f = \sigma(w_f[h^{(t-1)}, a^{(t)}] + b_f) \quad (6)$$

$$\Gamma_o = \sigma(w_o[h^{(t-1)}, a^{(t)}] + b_o) \quad (7)$$

The calculation of the three gates are similar to $\tilde{c}^{(t)}$. They act as switches to control output values. Γ_u , Γ_f and Γ_o are called update gate, forget gate, and output gate respectively.

$$c^{(t)} = \Gamma_u \odot \tilde{c}^{(t)} + \Gamma_f \odot c^{(t-1)} \quad (8)$$

$$h^{(t)} = \Gamma_o \odot \tanh(c^{(t)}) \quad (9)$$

where $h^{(t)}$ is the final output of the hidden layer at the current time step. It is affected by three gates, which represent the trade-offs of the past, current and total output respectively. The $\odot$ operator is the Hadamard product. Based on abstract features, stack bidirectional LSTM is used to design the proposed architecture. Stack LSTM refers to a multi-layer LSTM, and bidirectional LSTM (Bi-LSTM) is to perform LSTM operation simultaneously in both directions in the original sequence. It means that $h^{(t)}$ is the concatenate vector of both directions outputs, $\overrightarrow{h}^{(t)}$ and $\overleftarrow{h}^{(t)}$, when using the Bi-LSTM. As the contextual information contains both the front and the back [37].

$$h^t = \begin{bmatrix} \overrightarrow{h}^t & \overleftarrow{h}^t \end{bmatrix} \quad (10)$$

3.3.3. Classification model

Fig. 5 shows the classification model of TSCRNN of TSCRNN. The classification model is composed of three parts, the first part includes the convolution related layers. This part contains four different layers, comprising 1D-CNN, BatchNormalization, ReLU, and MaxPooling.

The filter size of 1D-CNN is set to 3, the stride is 1, the padding is 1. The batch normalization [38] is used to standardize the output of the last layer, which makes gradient descent easier. At the end of this block, MaxPooling is applied to downsample the data.

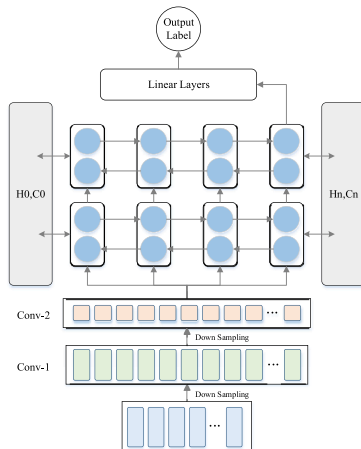


Fig. 5. The classification model in TSCRNN.

The second part includes the layers of RNN. The dimension of the hidden layer of the Bi-LSTM unit is set to 256. However, the actual output dimension is 512, for it concatenates the outputs of both directions. Bi-LSTM needs to retain the output of each time step except the last layer because of the stack architecture we used. Besides, to alleviate the phenomenon of overfitting, the dropout layer is added after Bi-LSTM, and the dropout rate is set to 0.5. The last part is a fully connected layer, which inputs a 512-dimensional vector and outputs a 16-dimensional vector. The final output dimension is the same as the number of categories corresponding to scenarios. Finally, Softmax is used to calculate the prediction probability of each type. Detailed architecture and parameters of the classification model are shown in Table I, where F is filter size and S is stride and the total parameters are 2,897,104.

It extracts abstract features before learning temporal characteristics, that can reduce the number of parameters. The classification model combines the speed of CNN and the time-sensitivity of RNN, which makes it more lightweight while retaining both advantages.

In the training processing, the batch size is set to 128 and the optimizer is Adam. Besides, we use the learning rate scheduling technique to help the model converge better. First of all, in the first 30 epochs, the learning rate is set to $1e-3$, as a larger learning rate can accelerate the convergence in the early phase of training. After that, the learning rate will be reduced by a fixed multiple of 0.3 iteratively, training a total of 20 epochs. Finally, the best performance model is obtained by early stop.

Algorithm 1 is the pseudo-code of the classification model of TSCRNN, describing how a vector of traffic is identified by the classification model.

4. Experiments

4.1. Experimental dataset

We implemented most experiments based on the ISCX-Tor2016 dataset, which was collected by Lashkari on the network of New Brunswick University (UNB) [12]. ISCX-Tor2016 contains two kinds of labels. One is traffic application, including AIM chat, Skype, Youtube, etc. The other is traffic service type, including Chat, Audio, FTP, etc., as shown in Table II. Two versions of each type are collected in a regular network and a tor network at the same time.

To validate the universality of TSCRNN, we also implemented experiments based on other datasets. One of them is the VPN-nonVPN dataset [27]. The details of the VPN-nonVPN dataset are shown in Table III.

Another dataset is the USTC-TFC2016 dataset, containing ten categories of benign traffic and ten categories of malicious traffic [39], as shown in Table IV.

All the datasets mentioned above were collected from the real network environment.

Table I
Parameters of the classification model.

Block	Layer Name	F	S	Output	Parameter
Conv1	Conv1d-1	3	1	(64, 1500)	2,944
	BatchNorm1d-2	-	-	(64, 1500)	128
	ReLU-3	-	-	(64, 1500)	-
	MaxPool1d-4	2	2	(64, 750)	-
Conv2	Conv1d-5	3	1	(64, 750)	12,352
	BatchNorm1d-6	-	-	(64, 750)	128
	ReLU-7	-	-	(64, 750)	-
	MaxPool1d-8	2	2	(64, 325)	-
LSTM	Bi-Lstm-9	-	-	(64, 512)	-
	Dropout-10	-	-	(64, 512)	2,873,344
	Bi-Lstm-11	-	-	(512,)	-
	Dropout-12	-	-	(512,)	-
FC	Linear-13	-	-	(16,)	8,208
Softmax	Softmax-14	-	-	(16,)	-

Table II

The description of the tor-nontor dataset.

Type	Labels
Tor	Audio, Browsing, Chat, Email, FTP, P2P, Video, VoIP
nonTor	torAudio, torBrowsing, torChat, torEmail, torFTP, torP2P, torVideo, torVoIP

Table III

The description of the VPN-nonVPN dataset.

Type	Labels
VPN	Chat, Email, FTP, P2P, Streaming, VoIP
nonVPN	vpnChat, vpnEmail, vpnFTP, vpnP2P, vpnStreaming, vpnVoIP

Table IV

The description of USTC-TFC2016 dataset.

Type	Labels
Benign	BitTorrent, Facetime, FTP, Gmail, MySQL, Outlook, Skype, SMB, Weibo, WorldOfWarcraft
Malware	Cridex, Geodo, Htbot, Miuref, Neris, Nsis-ay, Shifu, Tinba, Virut, Zeus

4.2. Evaluation metric

We used four metrics to evaluate the classification performances of different methods, including accuracy (*AC*), precision (*PR*), recall (*RC*), and *F1*. For a multi-classification problem, each class and the remaining samples are considered as a binary classification at first, which makes the precision and recall of each type can be calculated directly.

$$\text{Accuracy (AC)} = \frac{TP + TN}{TP + TN + FP + FN} \quad (11)$$

$$\text{Precision (PR)} = \frac{TP}{TP + FP} \quad (12)$$

$$\text{Recall (RC)} = \frac{TP}{TP + FN} \quad (13)$$

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (14)$$

where *TP*, *FP*, *TN*, *FN* refer to true positive, false positive, true negative, false negative respectively. To compare the quality of multiple indicators jointly, the harmonic average *F1* is used to unify both *PR* and *RC* into a single metric.

Since it is a multi-classification problem, it is not convenient to use *PR*, *RC*, *F1* for a single type. Therefore, Macro Average [40] is used to calculate the mean value of precision, recall, *F1* of each category as follows:

$$\text{Macro} - F1 = \frac{\sum_{i \in \text{classes}} F1_i}{N} \quad (15)$$

4.3. Experimental results and analysis

The experimental hardware platform is a computer equipped with a CPU of ADM RYZEN 3700 $\times$ 3.6GHz and a 16 GB of memory. The GPU accelerator is NVIDIA GeForce RTX 2070S. The framework runs on Python 3.7, and the deep learning platform is Pytorch 1.4.0. SplitCap (version 2.1) is used in the preprocessing phase, and the binary output stream is read by Scapy.

To test the performance of TSCRNN, four scenarios of different classification tasks were proposed in experiments as shown in Table V. Scenario A is a 2-classification to indicate whether the traffic flow is tor or normal traffic. Both scenario B and scenario C are 8-classification

tasks. Scenario B is to distinguish between eight normal traffic services like audio, browsing, etc. Scenario C is similar to B, while its target labels are eight traffic services of the tor version. Scenario D mixes all the sixteen types mentioned in scenario B and scenario C to perform the 16-classification task.

4.3.1. Selections of preprocessing strategy

In the preprocessing phase, different strategies were chosen, including the selection of sampling methods and the settings of packet parameters.

For sampling strategies, there were three methods tested, including Random Sampling, Fixed Step Sampling, and Mixed Sampling, and Table VI shows the *Marco* – *F1* of all scenarios affected by each of them.

As shown in Table VI, sampling strategies of both Fixed Step sampling and Mixed sampling have slightly improved the performance of scenario B and scenario D. However, the disparities are too small to be noticed, which implies that the traffic classification of middle flow sampling is not sensitive to sampling strategies. In other words, it has robustness in this respect.

Furthermore, we need to focus on the settings of packet parameters. From the distribution of packet number (Fig. 4 (a)) in a flow and packets length (Fig. 4 (b)), it can be observed that they are mainly distributed in (0, 15) and in (0, 1500) respectively. Consequently, the numbers of packets are selected as 5, 10, and 15, while the packet lengths are selected as 500, 1000, and 1500. And a grid search is performed for all of them.

The Table VII showed the performances with different packet parameter settings in all scenarios, including the packet number of each flow and packet length. It indicates that the improvement of packet length on the *F1* value is not obvious. Even after the packet length increases to more than 1000 bytes, there are some weak negative effects. It is speculated that the longer packet length is, the more information is obtained and the more noise is introduced at the same time.

Another packet parameter is the number of packets in each flow, which has a great impact on the performance. That is, with the increase of packets of each flow, the *F1* value of each scenario has a certain degree of improvement. This is because more packets can help the model to capture the time-related features.

It can be seen that the best performance emerged when the packet number is set to 15 and the packet length is set to 500, which demonstrates that the most valid information exists in the front part of the packet. However, considering the effect of packet length selections is weak which is also affected by training randomness, so the parameter of packet length is set to 1500 (consistent with MTU) without loss of generality. In subsequent experiments, the default sampling strategy is random sampling, while the number of packets is set to 15.

4.3.2. Experimental results

As shown in Table VIII and Fig. 6, TSCRNN performs well in all four scenarios. It is worth noting that all metrics of the whole types use macro-average and the same as below.

4.3.3. Comparison with related works

In [12], The author used machine learning methods to perform binary classifications and eight classification tasks of Tor traffic. Since the data imbalance problem of the binary classification task is not dealt with in the paper, the experimental results cannot reflect the real

Table V

The experimental description.

Scenario	Description Classifier
Scenario A	Tor and Non-Tor traffic identification 2-class
Scenario B	Regular encrypted traffic classification 8-class
Scenario C	Tor protocol encapsulated traffic classification 8-class
Scenario D	Encrypted traffic classification 16-class

Table VI

The Macro-F1 values of all scenario with different sampling strategies (F1).

Strategy	Scenario A	Scenario B	Scenario C	Scenario D
Random	0.979	0.979	0.909	0.948
Fixed Step	0.980	0.995	0.909	0.949
Mixed	0.979	0.996	0.908	0.947

Table VII

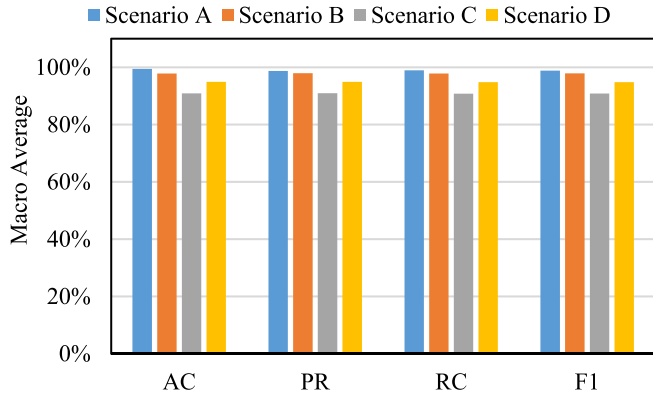
The Macro-F1 values of all scenario with different packet parameters (F1).

Strategy	Scenario A	Scenario B	Scenario C	Scenario D
5, 500	0.983	0.992	0.888	0.939
5, 1000	0.984	0.993	0.902	0.936
5, 1500	0.981	0.987	0.899	0.934
10, 500	0.979	0.992	0.909	0.947
10, 1000	0.981	0.990	0.913	0.946
10, 1500	0.979	0.979	0.912	0.940
15, 500	0.983	0.993	0.913	0.951
15, 1000	0.981	0.991	0.906	0.949
15, 1500	0.979	0.979	0.909	0.948

Table VIII

The experimental results in all scenarios.

Scenario	AC	PR	RC	F1
Scenario A	0.994	0.987	0.990	0.988
Scenario B	0.979	0.979	0.979	0.979
Scenario C	0.909	0.910	0.908	0.909
Scenario D	0.950	0.949	0.948	0.948

**Fig. 6.** The experimental results in all scenarios.

performance. Therefore, only the Tor eight classification task is compared as shown in Table IX. It can be seen from Table IX that TSCRNN performs better than machine learning methods, and its Precision and Recall reach 97.9% and 97.8%.

There are few methods based on deep learning on the Tor-nonTor dataset, so we implemented the other two methods based on deep learning to compare with TSCRNN.

One-dim CNN is mainly based on spatial features, and the model design refers to the paper [1]. Time Series mainly considers the time step characteristics of packets, whose design is based on LSTM. It can be seen

Table IX

Comparison of TSCRNN and machine learning methods in Tor eight classification task.

Metric	TSCRNN	RandomForest	C4.5	KNN
Precision	0.979	0.842	0.797	0.704
Recall	0.978	0.836	0.798	0.707

from Table X that the performance of TSCRNN based on spatiotemporal features in Scenario A, B, and D is better than other deep learning methods, and only slightly worse in Scenario C.

In recent years, many studies have conducted experiments of encrypted traffic classification on VPN-nonVPN dataset and proposed state of the art methods based on deep learning. To better compare with these methods, we conduct experiments on the VPN-nonVPN dataset as well.

Among them, One-dim CNN was proposed by Wang [26], and for the first time, CNN was used to capture the spatial characteristics of the traffic. Haipen Yao. et al [41] proposed HAN and Attention Based LSTM, which are based on time characteristics and combined with the attention mechanism. Deep Packet is proposed in the paper [7].

It can be seen from Table XI that TSCRNN has achieved excellent performance in the VPN-nonVPN dataset, and only Scenario B performed poorly.

4.3.4. Experiments of 16-classification

Further experiments were implemented in the most complex task in scenario D, that is, the sixteen classification task. As shown in Table XII, TSCRNN performs well in all types of traffic except Email and Chat. One reason is that there is not sufficient raw data of Email and Chat in the dataset. Another reason is that these two types of traffic are difficult to identify through multiple experiments, due to they are both short-lived data stream and similar in time distribution.

As shown in Fig. 7, it can readily find from the confusion matrix that the reason why the Email and Chat traffic have poor performance is that they are easily confused with each other. That is, 26% of Email flows are incorrectly identified as Chat, while 12% of Chat samples are predicted as Email. Such a phenomenon appeared in a series of experiments, which illustrates that the spatial and temporal distributions of both of them are similar. Fig. 8 visualizes the detailed experimental data for convenient observations.

4.3.5. Visualization of traffic

To verify whether the model correctly learns the spatiotemporal features, the activation on each byte in each flow is visualized as well. Fig. 9 shows one way to visualize traffic, which stretches the sample with the dimension of $1 \times 15 \times 1500$ to a one-dimensional array with the dimension of $22,500 \times 1$, regarding the bytes as the strength of the signal on each position with the range (0, 255).

It is easy for our humans to find that different service types of traffic in Fig. 10 have different distribution characteristics. Besides, compared with Fig. 9 (a) and Fig. 9 (b), it can be found that the distinction becomes more obvious after tor network encryption. That is why TSCRNN performed better on tor traffic than those of normal traffic in multiple experiments. Furthermore, it can be found that the presentations of Email and Chat are quite similar.

On the other hand, we visualized the activation on each point in the traffic flow of the classification model. The red area in Fig. 10 indicates the interest of the model in each part of the flow. The loss function is used to calculate the gradient on the input. And the values of gradient represent the activations on each byte, referring to their contribution to the final prediction.

In Fig. 10, four samples were randomly chosen. From the perspective of the overall flow, the interest region is more concentrated on the first few packets in a flow. And from the perspective of each packet, the model focuses on the front bytes of the packet. This is in line with our

Table X

Comparison of TSCRNN and deep learning methods in tor-nonTor dataset (F1).

Method	Scenario A	Scenario B	Scenario C	Scenario D
One-dim CNN	0.981	0.957	0.901	0.927
Time Series	0.979	0.941	0.915	0.925
TSCRNN	0.988	0.979	0.909	0.948

Table XI

Accuracy comparison of TSCRNN and deep learning methods in VPN-nonVPN dataset.

Method	Scenario A	Scenario B	Scenario C	Scenario D
One-dim CNN	0.990	0.818	0.986	0.866
HAN	0.995	0.851	0.929	0.895
Attention Based LSTM	0.997	0.893	0.948	0.912
Deep Packet	0.992	0.868	0.923	0.895
TSCRNN	0.997	0.864	0.965	0.917

Table XII

The detailed experimental results of sixteen classification task.

Type	PR	RC	F1
Audio	0.944	0.948	0.946
Browsing	0.949	0.970	0.960
Chat	0.733	0.822	0.775
Email	0.839	0.717	0.773
FTP	0.996	0.988	0.992
P2P	0.996	0.992	0.994
Video	0.922	0.901	0.912
VoIP	0.919	0.954	0.936
torAudio	0.981	0.966	0.973
torBrowsing	0.977	0.977	0.977
torChat	0.996	0.989	0.993
torEmail	0.996	0.975	0.985
torFTP	0.992	0.996	0.994
torP2P	0.959	0.992	0.976
torVideo	0.985	0.996	0.991
torVoIP	1.000	0.988	0.994
Macro-Avg	0.949	0.948	0.948

expectations that the model has captured spatiotemporal features of traffic flow correctly.

4.3.6. Experiments on other datasets

TSCRNN has performed well in the Tor-nonTor dataset. To verify its versatility, TSCRNN was applied to other datasets. Similarly, raw traffic of the other two datasets is input into TSCRNN and the labels are output, with the same preprocessing process, model, and parameter settings. The first dataset is the VPN-nonVPN dataset. The detailed experimental

results of the twelve classification tasks of the VPN-nonVPN dataset will be reported in Table XIII.

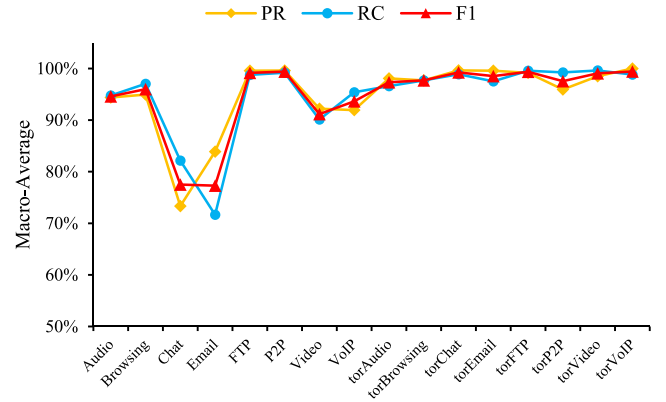
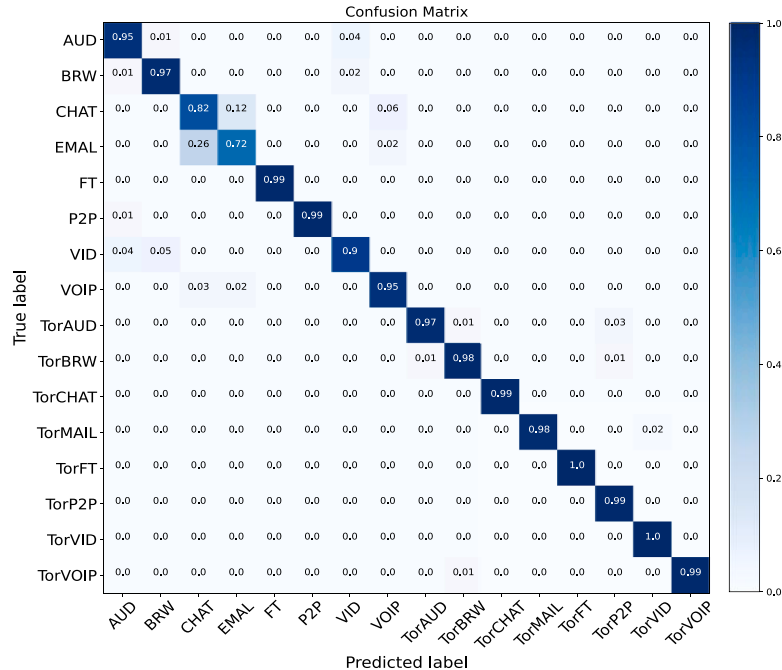
Furthermore, we validated the performance of TSCRNN on the USTC-TFC2016 datasets, detecting malicious traffic, which is different from the other two datasets captured by UNB. Table XIV shows the experimental results of a 20-classification task in USTC-TFC2016. TSCRNN achieves the F1 values up to 90% in mostly all categories, and even reaches 100% in some classes.

4.3.7. Summary of performance analysis

The accuracies of TSCRNN in Scenario A, B, C, D are 99.4%, 97.9%, 90.9%, 95.5% severally, while the values of F1 are 98.8%, 97.9%, 90.9%, and 94.8%. Experimental data, confusion matrix, and activation visualization demonstrates that TSCRNN can better learn the spatio-temporal features of the traffic flows.

TSCRNN was applied to the other two datasets, ISCXVPN2016 and USTC-TFC2016, without any special settings. The values of F1 of both entire datasets reached 92.6% and 98.7% respectively. It proves that TSCRNN is universal, not limited to specific scenarios.

To sum up, TSCRNN performs well in most classification tasks of

**Fig. 8.** The detailed experimental results of scenario D.**Fig. 7.** The confusion matrix of the sixteen-classification task.

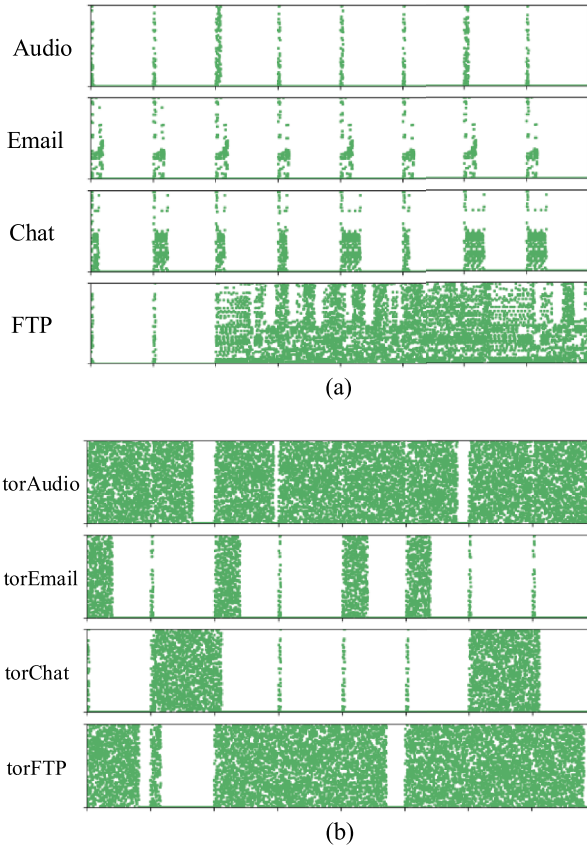


Fig. 9. The visualization of traffic flows. (a) are samples of normal traffic, and (b) are the samples of tor traffic.

network traffic.

5. Conclusion

Network encryption traffic classification is one of the essential tasks in systems of IIoT, such as networking, communication, security, and information processing. In this paper, we proposed a novel approach TSCRNN to realize encrypted traffic classification. There were pre-processing module and classification module in the framework of TSCRNN, which was input raw traffic file and outputted the label of prediction.

To evaluated TSCRNN, four distinct scenarios were set for

experiments. The main experiments were carried out on the Tor-nonTor dataset. TSCRNN outperforms other classification methods based on machine learning and deep learning. The reason is that TSCRNN considers the temporal and spatial characteristics at the same time. TSCRNN uses CNN to extract abstract features and then learned the temporal characteristics based on those low-dimensional feature maps by stack Bi-LSTM.

Furthermore, TSCRNN is validated on other datasets without any special settings, which achieves appropriate performance as well. It means that the traffic service perception and identification mechanism provided by TSCRNN will be helpful to IIoT, such as communication, network, information processing, and security.

In the future, we will focus on the following three aspects.

- The traffic datasets are always unbalanced. Therefore, generative models like the autoencoder or the generative adversarial network (GAN) can be used to solve this problem.
- There is currently no unified preprocessing scheme in the field of traffic classification. Different preprocessing methods can correspond to different levels of features, such as byte features, packet time-related features, statistical features, etc. Multi-level features fusion can be used to further improve performance.
- The labeling of real-world traffic is still a challenge in this field. Unsupervised learning and semi-supervised learning may help to classification with limited labeled samples.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence

Table XIII

The detailed experimental results of VPN-nonVPN dataset.

Type	PR	RC	F1
Chat	0.762	0.851	0.804
Email	0.842	0.780	0.810
FTP	0.943	0.892	0.917
P2P	0.977	0.955	0.966
Stream	0.965	0.935	0.950
VOIP	0.843	0.913	0.877
vpnChat	0.967	0.972	0.970
vpnEmail	0.963	0.987	0.975
vpnFTP	0.957	0.941	0.949
vpnP2P	0.972	0.965	0.968
vpnStream	0.950	0.945	0.948
vpnVOIP	0.985	0.977	0.981
Macro-Avg	0.927	0.926	0.926

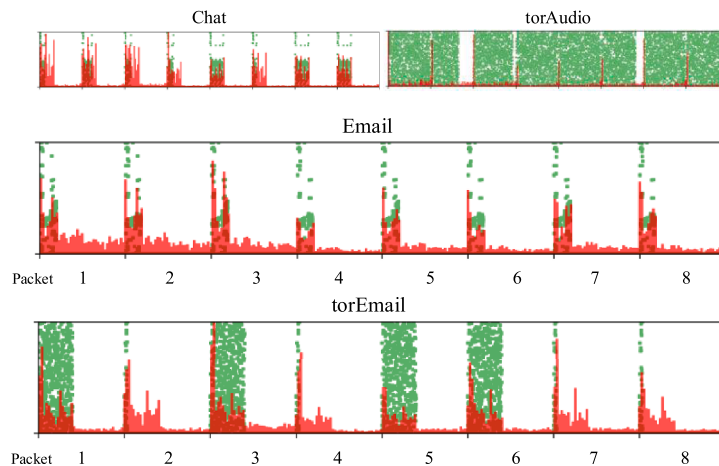


Fig. 10. The activations on each byte of traffic flow.

Table XIV

The detailed experimental results of USTC-TFC2016 dataset.

Type	PR	RC	F1
Cridex	1.000	1.000	1.000
Geodo	1.000	1.000	1.000
Htbot	1.000	1.000	1.000
Miuref	1.000	0.996	0.998
Neris	0.892	0.848	0.870
Nsis-ay	0.996	0.972	0.984
Shifu	0.996	1.000	0.998
Tinba	0.996	0.996	0.996
Virut	0.862	0.921	0.891
Zeus	1.000	1.000	1.000
BitTor	1.000	1.000	1.000
Facetime	1.000	1.000	1.000
FTP	1.000	1.000	1.000
Gmail	1.000	1.000	1.000
MySQL	1.000	1.000	1.000
Outlook	1.000	1.000	1.000
Skype	1.000	1.000	1.000
SMB	0.996	1.000	0.998
Weibo	1.000	0.996	0.998
Wow	1.000	1.000	1.000
Macro-Avg	0.987	0.986	0.987

Algorithm 1

Classification model.

Input: $x, x \in \mathbb{R}^{nk}$, referring to a vector of flow, which containing n packets and dimension of each packet is k
Output: The probabilities of each type
1 $a = x$;
2 Downsampling through N_{cnn} convolutional layers, including <i>Conv1D</i> , <i>BatchNormalization</i> , <i>MaxPooling</i> ;
3 for $i = 1; i \leq N_{cnn}; i++$ do
4 $a = \text{Conv1D}(a)$;
5 $a = \text{BatchNormalization1D}(a)$;
6 $a = \text{MaxPooling}(a)$;
8 end for
9 Learning time-series features through N_{lstm} layers of BiLSTM;
10 for $i = 1; i \leq N_{lstm}; i++$ do
11 Applying <i>LSTM</i> at each time step in both directions;
12 for $a^{(t)}$ in a do
13 $\vec{h}^{(t)} = \text{LSTM}_1(a^{(t)})$;
14 $\overleftarrow{h}^{(t)} = \text{LSTM}_2(a^{(t)})$;
15 $h^{(t)} = [\vec{h}^{(t)}, \overleftarrow{h}^{(t)}]$;
16 end for
17 $a = [h^{(1)}, h^{(2)}, \dots, h^{(t)}]$;
18 end for
19 $a = \text{FullConnet}(a^{(n)})$;
20 $probabilities = \text{Softmax}(a)$;
21 return $probabilities$;

the work reported in this paper.

Acknowledgments

We would like to thank the reviewers in advance for their comments to help us improve the quality of this paper. This work was supported by the National Natural Science Foundation of China under Grant 62072255.

References

- [1] H. Wang, Y. Wu, G. Min, W. Miao, A Graph Neural Network-based Digital Twin for Network Slicing Management, *IEEE Trans. Indust. Inform.* (2020).
- [2] S. Rezaei, X. Liu, Deep learning for encrypted traffic classification: An overview, *IEEE Commun. Mag.* 57 (5) (2019) 76–81.

- [3] H. Tahaei, F. Afifi, A. Asemi, F. Zaki, N.B. Anuar, The rise of traffic classification in IoT networks: A survey, *J. Netw. Comp. Appl.* 154 (2020), 102538.
- [4] J. Xie, F.R. Yu, T. Huang, R. Xie, J. Liu, C. Wang, Y. Liu, A survey of machine learning techniques applied to software defined networking (SDN): Research issues and challenges, *IEEE Commun. Surv. Tutor.* 21 (1) (2018) 393–430.
- [5] G. Aceto, D. Ciunzio, A. Montieri, A. Pescapé, Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges, *IEEE Trans. Netw. Serv. Manag.* 16 (2) (2019) 445–458.
- [6] S. Yu, M. Liu, W. Dou, X. Liu, S. Zhou, Networking for big data: A survey, *IEEE Commun. Surv. Tutor.* 19 (1) (2016) 531–549.
- [7] M. Lotfollahi, M.J. Siavoshani, R.S.H. Zade, M. Saberian, Deep packet: A novel approach for encrypted traffic classification using deep learning, *Soft Computing* 24 (3) (2020) 1999–2012.
- [8] S. Yu, W. Zhou, W. Jia, S. Guo, Y. Xiang, F. Tang, Discriminating DDoS attacks from flash crowds using flow correlation coefficient, *IEEE Trans. Parallel Distribut. Syst.* 23 (6) (2011) 1073–1080.
- [9] Y. Wu, Cloud-edge orchestration for the internet-of-things: Architecture and ai-powered data processing, *IEEE Internet of Things J.* (2020).
- [10] P. Ducange, G. Mannara, F. Marcelloni, R. Pecori, M. Vecchio, A novel approach for internet traffic classification based on multi-objective evolutionary fuzzy classifiers, in: 2017 IEEE international conference on fuzzy systems (FUZZ-IEEE), IEEE, 2017, pp. 1–6.
- [11] G. Aceto, D. Ciunzio, A. Montieri, A. Pescapé, Toward effective mobile encrypted traffic classification through deep learning, *Neurocomputing* 409 (2020) 306–315.
- [12] A.H. Lashkari, G.D. Gil, M.S.I. Mamun, A.A. Ghorbani, Characterization of tor traffic using time based features, in: ICISSP 2017 - Proceedings of the 3rd International Conference on Information Systems Security and Privacy 2017-Janua (September), 2017, pp. 253–262, <https://doi.org/10.5220/0006105602530262>.
- [13] Y. Qi, L. Xu, B. Yang, Y. Xue, J. Li, Packet classification algorithms: From theory to practice, in: IEEE INFOCOM 2009, IEEE, 2009, pp. 648–656.
- [14] A. Madhukar, C. Williamson, A longitudinal study of P2P traffic classification, in: 14th IEEE International Symposium on Modeling, Analysis, and Simulation, IEEE, 2006, pp. 179–188.
- [15] J. Sherry, C. Lan, R.A. Popa, S. Ratnasamy, Blindbox: Deep packet inspection over encrypted traffic, in: Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, 2015, pp. 213–226.
- [16] D. Bonfiglio, M. Mellia, M. Meo, D. Rossi, P. Tofanelli, Revealing skype traffic: when randomness plays with you, in: Proceedings of the 2007 conference on Applications, technologies, architectures, and protocols for computer communications, 2007, pp. 37–48.
- [17] S. Yu, G. Zhao, W. Dou, S. James, Predicted packet padding for anonymous web browsing against traffic analysis attacks, *IEEE Trans. Inform. Forens. Secur.* 7 (4) (2012) 1381–1393.
- [18] H. Liu, Z. Wang, Y. Wang, Semi-supervised encrypted traffic classification using composite features set, *J. Netw.* 7 (8) (2012) 1195.
- [19] B. Anderson, S. Paul, D. McGrew, Deciphering malware's use of TLS (without decryption), *J. Comp. Virol. Hack. Tech.* 14 (3) (2018) 195–211.
- [20] A. Fahad, Z. Tari, I. Khalil, I. Habib, H. Alnuweiri, Toward an efficient and scalable feature selection approach for internet traffic classification, *Comp. Netw.* 57 (9) (2013) 2040–2057.
- [21] A. Moore, D. Zuev, M. Crogan, Discriminators for use in flow-based classification, *Tech. Rep.* (2013).
- [22] B. Yamansavascular, M.A. Guvensan, A.G. Yavuz, M.E. Karsligil, Application identification via network traffic classification, in: 2017 International Conference on Computing, Networking and Communications (ICNC), IEEE, 2017, pp. 843–848.
- [23] R. Alshammari, A.N. Zincir-Heywood, Can encrypted traffic be identified without port numbers, IP addresses and payload inspection? *Comp. Netw.* 55 (6) (2011) 1326–1350.
- [24] J. Zhang, X. Chen, Y. Xiang, W. Zhou, J. Wu, Robust network traffic classification, *IEEE/ACM Trans. Netw.* 23 (4) (2014) 1257–1270.
- [25] E. Hodo, X. Bellekens, E. Iorkyase, A. Hamilton, C. Tachtatzis, R. Atkinson, Machine learning approach for detection of nontor traffic, in: Proceedings of the 12th International Conference on Availability, Reliability and Security, 2017, pp. 1–6.
- [26] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: 2017 IEEE International Conference on Intelligence and Security Informatics (ISI), IEEE, 2017, pp. 43–48.
- [27] G. Draper-Gil, A.H. Lashkari, M.S.I. Mamun, A.A. Ghorbani, Characterization of encrypted and VPN traffic using time-related features, in: ICISSP 2016 - Proceedings of the 2nd International Conference on Information Systems Security and Privacy, no. Icissp, 2016, pp. 407–414, <https://doi.org/10.5220/0005740704070414>.
- [28] V. Tong, H.A. Tran, S. Souihi, A. Mellouk, A novel quic traffic classifier based on convolutional neural networks, in: 2018 IEEE Global Communications Conference (GLOBECOM), IEEE, 2018, pp. 1–6.
- [29] T. Shapira, Y. Shavitt, Flowpic: Encrypted internet traffic classification is as easy as image recognition, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), IEEE, 2019, pp. 680–687.
- [30] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, J. Lloret, Network traffic classifier with convolutional and recurrent neural networks for Internet of Things, *IEEE Access* 5 (2017) 18042–18050.
- [31] W. Wang, Y. Sheng, J. Wang, X. Zeng, X. Ye, Y. Huang, M. Zhu, HAST-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection, *IEEE Access* 6 (2017) 1792–1806.

- [32] S.Rezaei, X.Liu, How to achieve high classification accuracy with just a few labels: A semi-supervised approach using sampled packets, arXiv preprint arXiv: 1812.09761 (2018).
- [33] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, *Nature* 521 (7553) (2015) 436–444. Google Scholar Google Scholar Cross Ref Cross Ref.
- [34] Y. Kim, Convolutional neural networks for sentence classification, *EMNLP* (2014).
- [35] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Computation* 9 (8) (1997) 1735–1780.
- [36] Y. Bengio, P. Simard, P. Frasconi, Learning long-term dependencies with gradient descent is difficult, *IEEE Trans. Neur. Netw.* 5 (2) (1994) 157–166.
- [37] X. Liu, J. You, Y. Wu, T. Li, L. Li, Z. Zhang, J. Ge, Attention-based bidirectional GRU networks for efficient HTTPS traffic classification, *Inform. Sci.* 541 (2020) 297–315.
- [38] S. Ioffe, C. Szegedy, Batch normalization: Accelerating deep network training by reducing internal covariate shift, in: *International conference on machine learning*, PMLR, 2015, pp. 448–456.
- [39] W. Wang, M. Zhu, X. Zeng, X. Ye, Y. Sheng, Malware traffic classification using convolutional neural network for representation learning, in: *2017 International Conference on Information Networking (ICOIN)*, IEEE, 2017, pp. 712–717.
- [40] C. Liu, W. Wang, M. Wang, F. Lv, M. Konan, An efficient instance selection algorithm to reconstruct training set for support vector machine, *Knowledge-Based Systems* 116 (2017) 58–73.
- [41] H. Yao, C. Liu, P. Zhang, S. Wu, C. Jiang, S. Yu, Identification of Encrypted Traffic Through Attention Mechanism Based Long Short Term Memory, *IEEE Transactions on Big Data* (2019).



Kunda Lin is currently working as a researcher for the Jiangsu Key Laboratory of Big Data Security & Intelligent Processing, Nanjing, China. His research interests include AI-based IoT optimization. He has a patent in the above field authorized by the State Intellectual Property Office of China as one of the key inventors. He has won three national and provincial competition awards, and ranked first in school algorithm competition. He won several scholarships during university. He used to work at Institute of Big Data Research at Yancheng. He is going to pursue his PhD degree major in Information networks in Nanjing University of Posts & Telecommunications, Nanjing, China.



Xiaolong Xuis currently a professor in the School of Computer Science, Nanjing University of Posts & Telecommunications, Nanjing, China. He is also working for the Jiangsu Key Laboratory of Big Data Security & Intelligent Processing. He received the B.E. in computer and its applications, M.E. in computer software and theories and Ph.D. degree in communications and information systems, in 1999, 2002 and 2008, respectively. He is a senior member of China Computer Federation. He teaches graduate courses and conducts research in areas of Cloud Computing, Big Data, Information Security and Novel Network Computing Technologies. As the leader of project teams, he has successfully completed a number of high-level research projects, including the projects sponsored by the National Science Fund of China. He has published more than 100 Journal and conference papers as the first or corresponding author and 5 books. He is authorized 52 patents by the State Intellectual Property Office of China as the first inventor. He was rated as excellent young professor of Jiangsu Province in 2014, selected as the high-level creative talents of Jiangsu province in 2015, and won the title of outstanding expert in the area of computer science and technology.



Honghao Gao received the Ph.D. degree in Computer Science and started his academic career at Shanghai University in 2012. Prof. Gao is currently with the School of Computer Engineering and Science, Shanghai University, China. He is also a Professor at Gachon University, South Korea. Prior to that, he was a Research Fellow with the Software Engineering Information Technology Institute of Central Michigan University (CMU), USA, and was also an Adjunct Professor at Hangzhou Dianzi University, China. His research interests include Software Formal Verification, Industrial IoT/Wireless Networks, Service Collaborative Computing, and Intelligent Medical Image Processing. He has publications in *IEEE TII*, *IEEE T-ITS*, *IEEE IoT-J*, *IEEE TNSE*, *IEEE TCCN*, *IEEE/ACM TCBB*, *ACM TOMM*, *IEEE TCSS*, *IEEE TETCI*, *IEEE Network*, and *IEEE JBHI*. Prof. Gao is a Fellow of IET, BCS, and EAI, and a Senior Member of IEEE, CCF, and CAAI. He is the Editor-in-Chief for *International Journal of Intelligent Internet of Things Computing (IJIITC)*, Editor for *Wireless Network (WINE)* and *IET Wireless Sensor Systems (IET WSS)*, and Associate Editor for *IET Software*, *International Journal of Communication Systems (IJCS)*, *Journal of Internet Technology (JIT)*, and *Journal of Medical Imaging and Health Informatics (JMIHI)*. Moreover, he has broad working experiences in industry-university-research cooperation. He is a European Union Institutions appoint external expert for reviewing and monitoring EU Project, is a member of the EPSRC Peer Review Associate College for UK Research and Innovation in the UK, and is also a founding member of IEEE Computer Society Smart Manufacturing Standards Committee.